

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Сыктывкарский лесной институт (филиал) федерального
государственного бюджетного образовательного учреждения
высшего образования «Санкт-Петербургский государственный
лесотехнический университет имени С. М. Кирова»
(СЛИ)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

CMS для парикмахерских и барбершопов «Сайтама»

Исполнитель

В. А. Решетняк

Руководитель практики

Н. А. Оганезова

Сыктывкар 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 ПОСТАНОВКА ЗАДАЧИ	6
2 АВТОМАТИЗИРУЕМЫЕ БИЗНЕС-ПРОЦЕССЫ	7
3 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И АРХИТЕКТУРНЫЕ ШАБЛОНЫ	8
3.1 Общая схема	8
3.2 Архитектурные шаблоны бэкенда	8
3.3 Архитектура фронтенда	9
4 ТРЕБОВАНИЯ ЗАКОНОДАТЕЛЬСТВА	11
5 ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ РЕАЛИЗАЦИИ	12
6 ДОПОЛНИТЕЛЬНЫЕ ИНСТРУМЕНТЫ РАЗРАБОТКИ	14
7 СТРУКТУРА БАЗЫ ДАННЫХ	16
7.1 Инфологическая модель	16
7.2 Даталогическая модель	17
8 ОПИСАНИЕ РАЗРАБОТАННОГО ПРИЛОЖЕНИЯ	18
8.1 Создание записи (бронирование)	18
8.2 Первый запуск системы (мастер /setup)	18
9 ПОДКЛЮЧЕНИЕ АНАЛИТИКИ	21
10 РАЗМЕЩЕНИЕ В СЕТИ ИНТЕРНЕТ	22
ЗАКЛЮЧЕНИЕ	23
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	24
ПРИЛОЖЕНИЕ А. Диаграммы последовательностей	26

СПИСОК СОКРАЩЕНИЙ И ТЕРМИНОВ

CMS – Content Management System – система управления содержимым сайта.

API – Application Programming Interface – программный интерфейс взаимодействия компонентов системы.

REST – Representational State Transfer – архитектурный стиль построения API поверх HTTP.

SPA – Single Page Application – одностраничное веб-приложение (фронтенд).

DTO – Data Transfer Object – объект переноса данных на границе API, отделённый от внутреннего представления в БД.

ORM – Object-Relational Mapping – отображение объектов языка программирования на строки реляционной таблицы.

JWT – JSON Web Token – формат токена авторизации, используемый для access-токена сессии.

TLS – Transport Layer Security – протокол шифрования канала связи (HTTPS).

CI – Continuous Integration – непрерывная автоматическая проверка (тесты, линтеры) при каждом изменении кода.

СУБД – система управления базами данных (в проекте – PostgreSQL 16).

БД – база данных.

ФТ – функциональное требование (карточки ФТ-001–ФТ-015 в техническом задании).

Точка сети – отдельная парикмахерская/барбершоп в составе сети одного владельца – собственный адрес, часы работы, мастера и записи.

Мастер – сотрудник точки, оказывающий услуги клиентам; в системе – роль пользователя.

ВВЕДЕНИЕ

Предприятия сферы парикмахерских услуг и барберинга всё чаще переходят от бумажных журналов и разрозненных таблиц к специализированным программным системам. Такие решения обеспечивают онлайн-запись клиентов, ведение расписания мастеров и автоматизацию отчётности. Отсутствие единого инструмента порождает типовые для отрасли проблемы: возникновение накладок в расписании (двойных записей), неявки клиентов из-за отсутствия напоминаний, сложности с дифференцированным учётом цен мастеров, а также отсутствие у руководства оперативной аналитики по выручке и загрузке заведений. Настоящий документ посвящён веб-приложению «Сайтама», автоматизирующему данные бизнес-процессы для отдельных салонов и сетевых барбершопов.

В работе формулируется решаемая задача, описываются автоматизируемые процессы, архитектура приложения, применённые паттерны проектирования и требования нормативно-правовой базы. Обосновывается выбор технологического стека (FastAPI [7], SQLAlchemy [8], PostgreSQL [12]), приводится схема базы данных, детально рассматривается логика ключевых пользовательских сценариев, интеграция аналитических модулей и развёртывание сервиса в сети Интернет.

Комплект проектной документации включает три сопутствующих документа:

- 1) Техническое задание, оформленное в соответствии с ГОСТ 34.602-2020 [1] и ГОСТ 19.201-78 [2], содержащее постановку задачи и перечень функциональных требований (ФТ-001–ФТ-015);
- 2) Руководство пользователя, описывающее пошаговые сценарии работы для четырёх ролей: клиента, мастера, администратора филиала и владельца сети;
- 3) Руководство администратора, регламентирующее процедуры развёртывания, эксплуатации и резервного копирования серверной инфраструктуры.

Практическая значимость результатов заключается в создании готовой к промышленному внедрению программной платформы, устраняющей издержки ручного учёта и человеческого фактора. Архитектура системы изна-

чально поддерживает мультифилиальность, что позволяет развёртывать её как в единичных парикмахерских, так и в разветвлённых сетях без внесения изменений в исходный код.

Структура работы включает введение, аналитическую часть, проектную часть, заключение, библиографический список и приложения.

1 ПОСТАНОВКА ЗАДАЧИ

Исходная проблема. Барбершоп «Сайтама», выбранный пилотным объектом внедрения, вёл запись клиентов и учёт услуг вручную (телефон, бумажный журнал, разрозненные записи) – это порождало типовые для отрасли проблемы: двойные записи одного мастера на одно и то же время, отсутствие напоминаний клиентам и, как следствие, неявки (no-show), отсутствие единого каталога услуг с ценами, различающимися по мастеру, и отсутствие у владельца оперативной картины по выручке и загруженности без ручного подсчёта.

Задача. Разработать веб-приложение, автоматизирующее полный цикл операционной деятельности барбершопа: онлайн-запись клиента без участия администратора, ведение каталога услуг с индивидуальным прайсом по мастерам, автоматические напоминания о визите, формирование отчётов по выручке и загруженности, и – что не было предусмотрено первоначальной редакцией технического задания, но было добавлено по ходу разработки – поддержку сети из нескольких точек с ролью владельца сети поверх администратора отдельной точки.

Измеримые цели (полный перечень с формулировками показателей – техническое задание, раздел 2.2): сокращение времени оформления записи клиентом до 2 минут; исключение двойных бронирований техническими средствами (ограничением целостности СУБД, а не организационно); снижение доли неявок за счёт автоматических напоминаний; поддержка индивидуального прайса услуги по мастеру; отказ от бумажных/табличных журналов в пользу единого хранилища данных; формирование отчёта по выручке и загруженности без участия разработчика.

Границы задачи. Вне рамок разработки остались: приём онлайн-оплаты (оплата очно, см. техническое задание, раздел 3.2), бухгалтерский и складской учёт, CRM-функции сверх базового профиля клиента (сегментация, маркетинговые рассылки, программы лояльности). Эти ограничения не изменились по ходу разработки.

2 АВТОМАТИЗИРУЕМЫЕ БИЗНЕС-ПРОЦЕССЫ

Система автоматизирует следующие бизнес-процессы, выделенные при обследовании предметной области (техническое задание, раздел 2.1):

- 1) онлайн-запись клиента (Зап-1) – выбор услуги, мастера, свободного времени и подтверждение записи без участия администратора; для незарегистрированного клиента запись создаёт учётную запись автоматически (см. раздел 8);
- 2) управление услугами и прайсом (Усл-2) – единый каталог услуг с ценой, индивидуальной по каждой паре «мастер – услуга»;
- 3) напоминания о записи (Увед-3) – автоматическая отправка напоминаний за 24 и за 2 часа до визита;
- 4) формирование отчётов (Отчёт-4) – выручка, загруженность мастеров, топ-услуги за произвольный период, без привлечения разработчика (раздел 9);
- 5) управление контентом сайта (Конт-5) – настройки, отображаемые на главной странице и странице мастеров;
- 6) управление сетью точек обслуживания (Точ-6) – открытие, закрытие, изменение адреса, телефона и часов работы каждой точки.

Процесс, добавленный по ходу разработки. Управление сетью точек обслуживания (Точ-6) не входило в первую редакцию технического задания – поддержка сети из нескольких точек (роль «владелец сети» шире «администратора точки», переключение между точками, назначение администратора конкретной точке) была добавлена по ходу разработки, и техническое задание обновлено вместе с кодом. Функции по ролям (клиент/мастер/администратор/владелец) с полным перечнем операций – руководство пользователя, разделы 3.4.1–3.4.4.

3 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И АРХИТЕКТУРНЫЕ ШАБЛОНЫ

3.1 Общая схема

Система – классическое клиент-серверное веб-приложение: одностраничное приложение (SPA) на Vue 3 [11], обращающееся к REST API на FastAPI [7], который работает с единственной базой данных PostgreSQL [12]. Слоистая архитектура бэкенда и то, куда в неё встраивается фронтенд, показаны на рисунке 1; топология развёртывания (Docker-контейнеры, реверс-прокси, TLS) – предмет отдельного документа (руководство администратора, раздел 1) и здесь не повторяется.

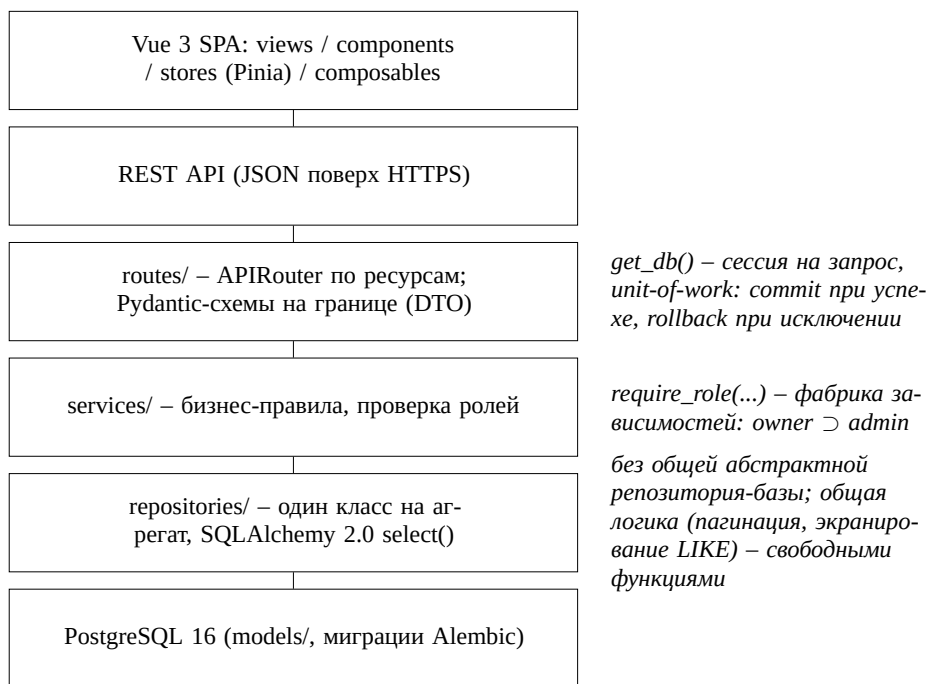


Рисунок 1 – Слоистая архитектура: от фронтенда до базы данных

3.2 Архитектурные шаблоны бэкенда

Backend (backend/app/) построен как строгая однонаправленная слоистая архитектура – routes → services → repositories → models, – с параллельным пакетом schemas/ Pydantic-схем [10], используемых только на границе запроса: сервисы всегда возвращают `XResponse.model_validate(orm_obj)`, поэтому «сырые» ORM-объекты никогда не попадают наружу через HTTP.

Внутри этой схемы использовано несколько именованных шаблонов проектирования:

- **DTO на границе (Data Transfer Object).** Разделение `schemas/` (что видит клиент API) и `models/` (как данные лежат в БД) – изменение внутреннего представления не ломает контракт API, и наоборот.
- **Repository.** Один класс-репозиторий на агрегат (`UserRepository`, `AppointmentRepository`, `SalonRepository` и т. д.) – вся работа с SQLAlchemy [8] сосредоточена в этом слое; сервисы не составляют SQL-запросы напрямую.
- **Dependency Injection через Depends.** Отдельного DI-контейнера нет – вся инъекция зависимостей идёт штатным механизмом FastAPI. `get_db()` – сессия БД на один запрос, реализующая шаблон **Unit of Work**: один `commit` при успешном завершении запроса, `rollback` при любом исключении, `db.close()` в `finally`.
- **Dependency-factory.** `require_role(*roles)` – функция, возвращающая функцию-зависимость; так собраны `get_current_client/master/admin/owner`. Поскольку `owner ⊃ admin` по бизнес-правилу, `get_current_admin` проверяет обе роли сразу.
- **Явное разрешение области видимости (salon scoping).** Хелпер `resolve_salon_scope(current_user, salon_id)` – не `Depends`, а обычная функция, вызываемая внутри уже защищённых ролю эндпоинтов: для `owner` отсутствие `salon_id` означает «вся сеть», для `admin` – доступ жёстко ограничен собственной точкой (несовпадение – код ответа 403).

3.3 Архитектура фронтенда

Frontend (`frontend/src/`) – Vue 3 на Composition API (`<script setup>` повсеместно, без TypeScript), Vue Router 4, управление состоянием – Pinia (шесть «setup-хранилищ» – `auth`, `salon`, `setup`, `siteContent`, `masterProfile`, `toast`). Маршрутизация – один файл `router/index.js` с ленивой загрузкой всех маршрутов и единым глобальным `beforeEach`-хуком, который, помимо проверки ролей (`meta.roles`), принудительно блокирует всё приложение мастером первого запуска `/setup`, пока тот не завершён (см. раздел 8.2).

Два решения на уровне фронтенда стоит выделить отдельно как архитектурные, не просто UI-детали:

– **Токен сессии – только в httpOnly cookie**, никогда не в localStorage/JS-переменной (снижает риск кражи токена через XSS). Обновление токена – через axios-интерцептор с *single-flight*: при 401 запускается один POST/auth/refresh, параллельные запросы дожидаются общего promise вместо каждый – своего обновления, и только при неудаче самого refresh срабатывает глобальный logout.

– **Тема оформления как настройка, а не константа.** Цвета бренда (brand/accent/ink/stone) объявлены в Tailwind не фиксированными значениями, а CSS custom properties, переключаемыми во время выполнения из настроек сайта, заданных администратором – один и тот же собранный фронтенд обслуживает точки с разным оформлением без пересборки.

4 ТРЕБОВАНИЯ ЗАКОНОДАТЕЛЬСТВА

Согласно техническому заданию (раздел 1.4), система должна соответствовать Федеральному закону № 152-ФЗ «О персональных данных» [5] и Федеральному закону № 149-ФЗ «Об информации, информационных технологиях и о защите информации» [6]; методология оценки достаточности реализованных мер защиты опирается на критерии ГОСТ Р ИСО/МЭК 15408-3-2008 [4]. Ниже – то, что из них фактически реализовано в коде.

- **Согласие на обработку персональных данных.** Форма регистрации требует явной отметки согласия (чекбокс со ссылкой на политику конфиденциальности) – отправка формы блокируется, пока согласие не отмечено; то же условие распространено на вход через VK ID.

- **Страница политики конфиденциальности** (маршрут /privacy) – десять разделов, соответствующих типовой структуре документа по 152-ФЗ (правовые основания, категории данных, цели обработки, права пользователя, раскрытие использования cookie и аналитики, контактные данные оператора).

- **Раскрытие использования cookie и аналитики** в §8 политики конфиденциальности вместе с фактом использования Яндекс.Метрики (раздел 9).

- **Хранение и удаление персональных данных.** Пароли не хранятся в открытом виде (хэш пароля, поле password_hash); удаление учётной записи при наличии связанных записей о визитах технически заблокировано ограничением ссылочной целостности (ON DELETE RESTRICT) – вместо удаления применяется мягкая блокировка (is_blocked).

5 ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ РЕАЛИЗАЦИИ

Выбранный технологический стек и обоснование каждого решения приведены в таблице 1; ссылки в столбце «Средство» указывают на официальную документацию соответствующего инструмента.

Таблица 1 – Технологический стек и обоснование выбора

Средство	Обоснование
FastAPI (Python) [7]	Асинхронный фреймворк с валидацией запросов через Pydantic «из коробки» и автогенерацией интерактивной документации API (OpenAPI/Swagger, /api/docs) без отдельного описания контракта вручную.
PostgreSQL 16 [12]	Единственная СУБД, рассмотренная всерьёз, – из-за ограничения целостности EXCLUDE USING gist: требование «0 двойных бронирований» (техническое задание, раздел 2.2) решено <i>на уровне схемы БД</i> , а не проверкой в коде приложения перед записью, которая при параллельных запросах подвержена состоянию гонки.
SQLAlchemy 2.0 [8] + Alembic [9]	Типизированный ORM (Mapped[...]) и версионизируемые миграции схемы вместо ручных SQL-скриптов.
Pydantic [10]	Валидация входных данных и сериализация ответов единым механизмом, общим с автогенерацией документации FastAPI.
Vue 3 (Composition API) [11] + Pinia	Компактный реактивный фронтенд без избыточного бойлерплейта Vuex/Redux – оправдано масштабом проекта.
Tailwind CSS	Единая цветовая и типографическая система без написания собственного CSS-фреймворка с нуля.

Продолжение таблицы 1

Средство	Обоснование
Docker Compose [14] + Caddy [13]	Воспроизводимое развёртывание и автоматический TLS-сертификат Let's Encrypt без ручной настройки nginx/certbot; подробности – руководство администратора, раздел 1.

Осознанно без очереди задач (Celery/Redis). Рассылка напоминаний реализована фоновым asyncio-циклом внутри самого процесса бэкенда, без отдельного воркера или брокера сообщений (техническое задание, раздел 3.3). Решение сознательное, а не сужение первоначального плана: система рассчитана ровно на один работающий экземпляр контейнера backend (руководство администратора, раздел 1), и при этом ограничении отдельная очередь задач с брокером сообщений добавляла бы эксплуатационную сложность без практической выгоды. Тот же аргумент применён к защите от перебора паролей и ограничению частоты запросов – оба реализованы в памяти процесса (“slowapi” с Limiter по IP и таблица login_attempts соответственно), без Redis.

6 ДОПОЛНИТЕЛЬНЫЕ ИНСТРУМЕНТЫ РАЗРАБОТКИ

Вспомогательные инструменты линтинга, тестирования и непрерывной интеграции, использованные при разработке, перечислены в таблице 2.

Таблица 2 – Инструменты разработки, тестирования и контроля качества

Инструмент	Назначение
ruff	Линтер Python (E,F,B,UP,SIM,C4).
ESLint 9 + eslint-plugin-vue	Линтер JS/Vue (flat config, vue/flat/essential).
Pyright	Проверка типов Python в режиме basic (локально).
pytest (unit + integration)	Модульные тесты (backend/tests) и отдельно помеченные интеграционные тесты (backend/tests/integration) на настоящем Postgres через httpx.AsyncClient – покрывают ограничение EXCLUDE, уникальность slug точки, область видимости по точкам сети, полный сценарий записи «от начала до конца».
Vitest + @vue/test-utils	Модульные тесты фронтенда (хранилища, composables).
Playwright + @axe-core/playwright	End-to-end тесты в браузере (Chromium) и автоматическая регрессия по доступности (accessibility).
GitHub Actions (CI)	Параллельные job'ы на каждый push/PR в main: тесты и линт бэкенда, интеграционные тесты бэкенда на реальном Postgres, сборка и линт фронтенда, unit- и e2e-тесты фронтенда, синтаксическая проверка docker-compose.yml.
pip-audit / npm audit	Проверка зависимостей на известные уязвимости как отдельные шаги CI.

Продолжение таблицы 2

Инструмент	Назначение
Dependabot	Еженедельные автоматические PR на обновление зависимостей (pip, npm, Docker-образы обоих Dockerfile, GitHub Actions).

7 СТРУКТУРА БАЗЫ ДАННЫХ

7.1 Инфологическая модель

Модель разделена на две схемы для читаемости: рисунок 2 – сущности бизнес-домена (точки, пользователи, мастера, услуги, записи, отзывы); рисунок 3 – сущности, обеспечивающие безопасность входа (сессии, токены, попытки входа), и настройки сайта.

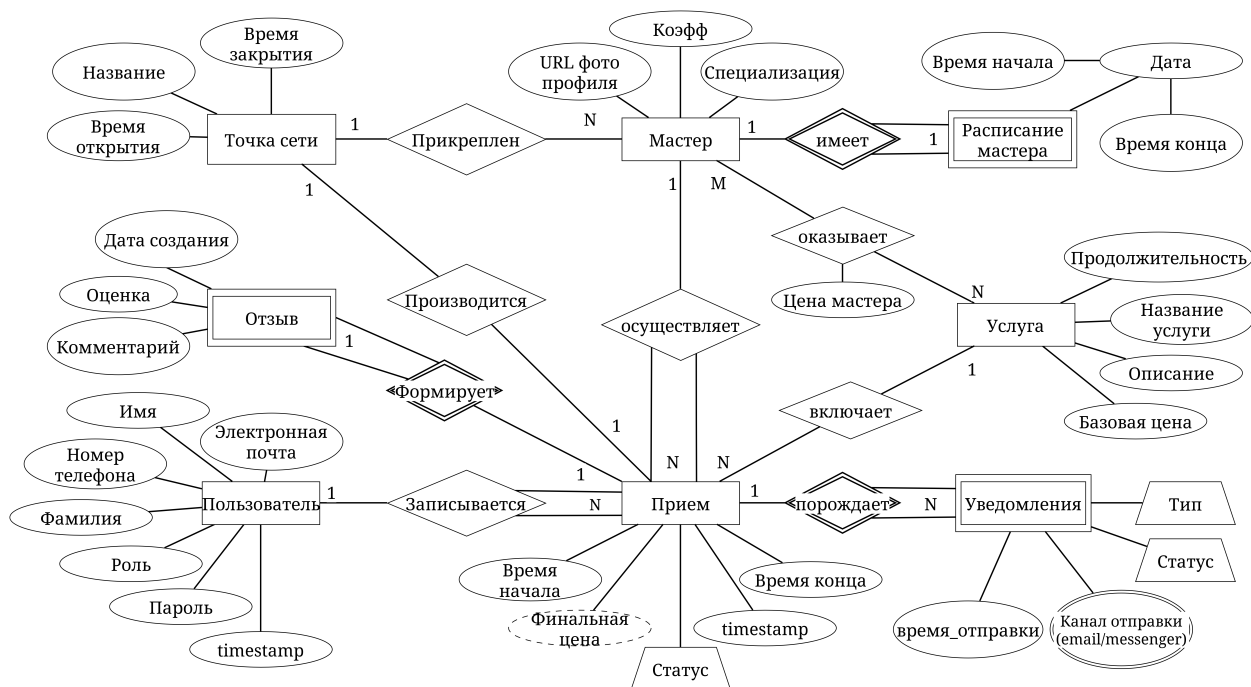


Рисунок 2 – Инфологическая модель – бизнес-домен (актуальная схема БД)

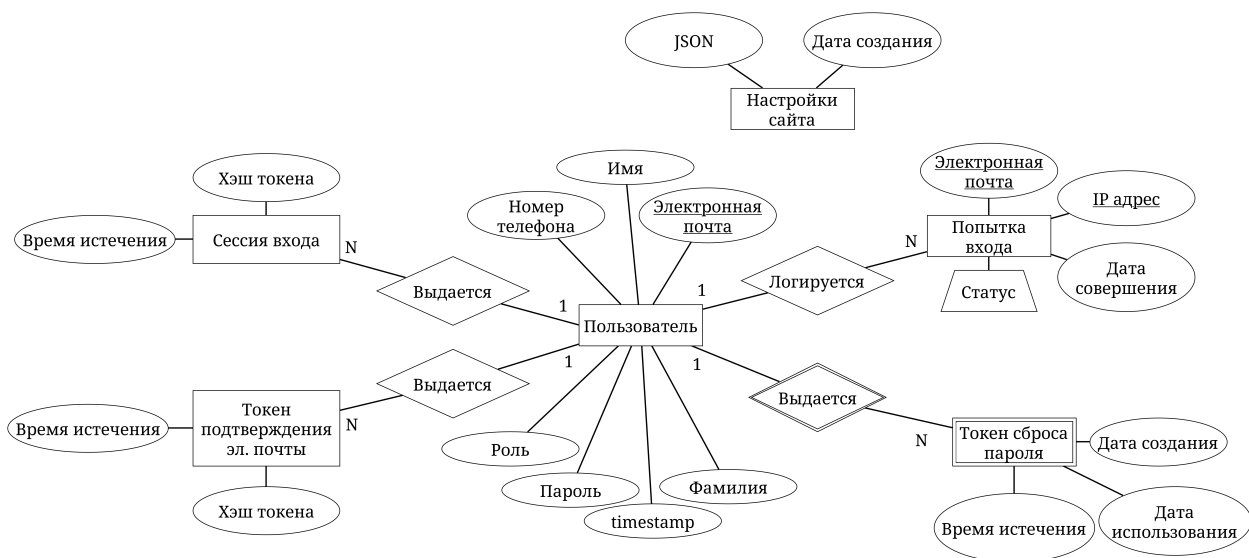


Рисунок 3 – Инфологическая модель – аутентификация и настройки сайта

7.2 Даталогическая модель

Фактическая схема БД насчитывает 13 таблиц (15 миграций Alembic [9] в директории backend/alembic/versions/):

- salons – точки сети (адрес, часы работы, slug);
- users – учётные записи всех ролей (client/master/admin/owner);
- masters – профиль мастера (специализация, коэффициент к цене);
- schedules – рабочее расписание мастера по дням недели;
- services – каталог услуг (цена, длительность);
- master_services – прайс услуги, индивидуальный по мастеру;
- appointments – записи на приём (EXCLUDE USING gist – защита от двойного бронирования на уровне СУБД);
- reviews – отзывы клиентов о завершённых записях;
- sessions – серверное хранилище refresh-сессий;
- password_reset_tokens, email_verification_tokens – одноразовые токены сброса пароля и подтверждения email;
- login_attempts – аудит попыток входа;
- site_settings – редактируемый контент сайта (единственная строка, JSONB).

Полный DDL-скрипт со столбцами, типами, первичными и внешними ключами – файл `datalogical_model.sql` в директории настоящего документа; скрипт готов к импорту в любой инструмент, читающий SQL DDL (`dbdiagram.io`, `DataGrip/DBever`, `pgAdmin ERD Tool`), для построения рисунка 4.

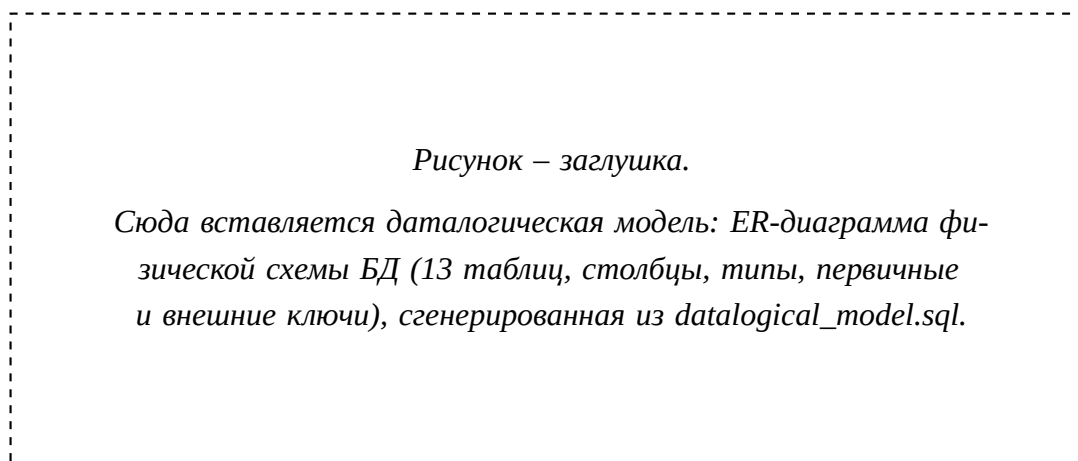


Рисунок 4 – Даталогическая модель (рисунок вставляется отдельно)

8 ОПИСАНИЕ РАЗРАБОТАННОГО ПРИЛОЖЕНИЯ

Полный перечень операций по каждой из четырёх ролей – руководство пользователя, разделы 3.4.1–3.4.4. Здесь – сквозные технические сценарии двух ключевых процессов на уровне «фронтенд → бэкенд → БД»; диаграммы последовательностей всех пяти автоматизируемых бизнес-процессов (раздел 2) – приложение А.

8.1 Создание записи (бронирование)

Технический сценарий создания записи на уровне «фронтенд → бэкенд → БД» показан на рисунке 5.

Повторная проверка доступности слота на шаге 6 на уровне СУБД, а не только на уровне кода перед вставкой, – прямая реализация цели «0 инцидентов двойного бронирования» из технического задания (раздел 2.2): при конкурентной попытке два запроса могут одновременно пройти проверку на шаге 1–4, но лишь один пройдёт INSERT на шаге 6, второй получит 409 от самой базы данных.

8.2 Первый запуск системы (мастер /setup)

Сценарий первого запуска, выполняемый мастером настройки /setup, представлен на рисунке 6.

Маршрут /setup проверяется единым глобальным `router.beforeEach`-хуком фронтенда перед *каждой* навигацией (раздел 3), поэтому попасть в остальную часть приложения, минуя этот сценарий, нельзя; после успешного шага 6 статус настройки самоблокируется навсегда (руководство администратора, раздел 3.3.4).

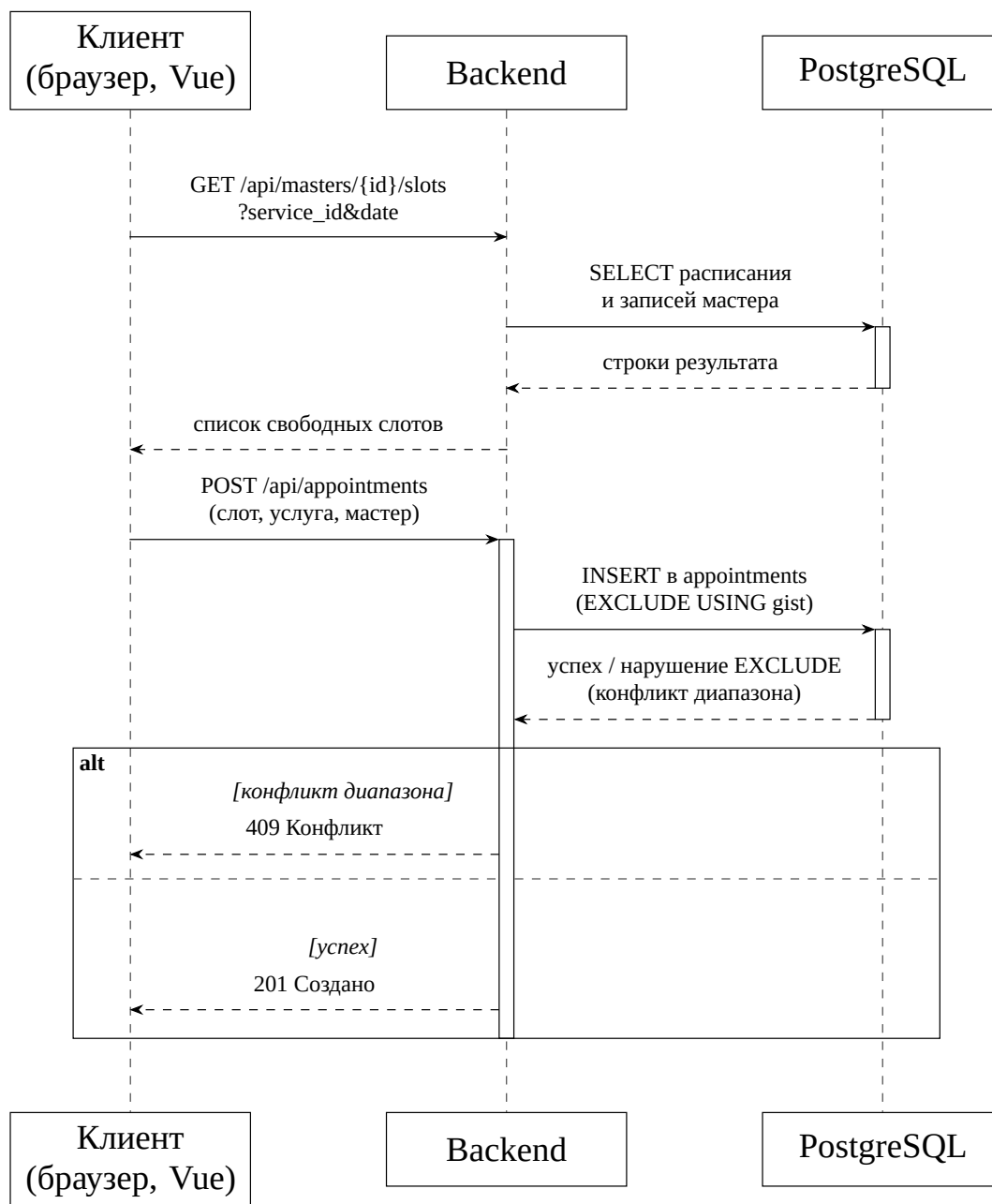


Рисунок 5 – Диаграмма последовательности: создание записи

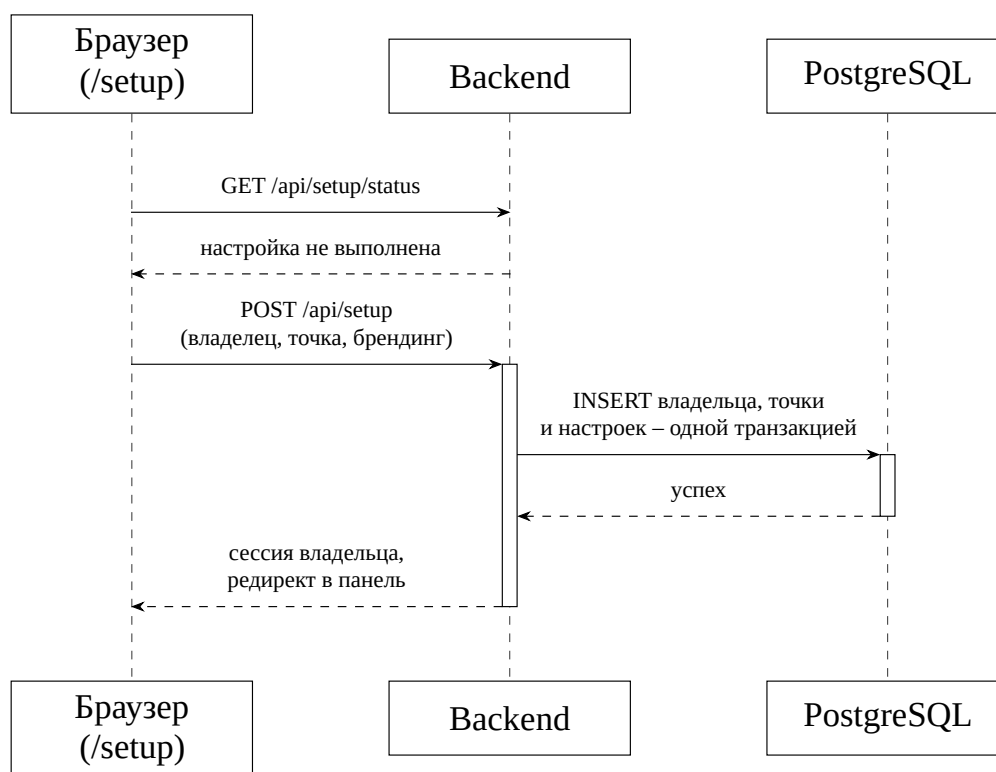


Рисунок 6 – Диаграмма последовательности: первый запуск (/setup)

9 ПОДКЛЮЧЕНИЕ АНАЛИТИКИ

В системе три независимых источника аналитики, которые не стоит путать между собой:

- **Веб-аналитика (Яндекс.Метрика) – подключена и активна.** Счётчик загружается отдельным статическим файлом (/metrika.js), а не инлайн-скриптом в index.html, – ради политики безопасности контента (CSP): script-src остаётся 'self' без исключения 'unsafe-inline'. Использование раскрыто в §8 политики конфиденциальности (раздел 4).
- **Мониторинг ошибок (Sentry) – код готов, отключён по умолчанию.** Инициализация присутствует и на бэкенде, и на фронтенде, но выполняется только при заданном SENTRY_DSN/VITE_SENTRY_DSN.
- **Бизнес-аналитика – собственные отчёты по данным приложения.** Раздел «Статистика» (KPI-плитки, график регистраций за 30 дней) и раздел «Отчёты» (выручка и загруженность по дням/мастерам/услугам с экспортом в Excel) – оба на данных из собственной БД проекта, без внешнего сервиса. Реализуют цель технического задания «формирование отчёта по выручке и загруженности без привлечения разработчика» (раздел 2.2); полное описание экранов – руководство пользователя, раздел 3.4.3.

10 РАЗМЕЩЕНИЕ В СЕТИ ИНТЕРНЕТ

Полный порядок развёртывания, включая перечень необходимых портов, подключение домена и TLS, первичную настройку и восстановление после сбоя, – предмет отдельного документа (руководство администратора, разделы 1–4) и здесь не повторяется. Кратко: система разворачивается как согласованный набор Docker-контейнеров [14] (база данных, backend, frontend, реверс-прокси Caddy [13] с автоматическим TLS-сертификатом Let’s Encrypt, ежедневное резервное копирование), наружу из этой топологии открыты только порты 80/443 контейнера Caddy, обновление выполняется вручную одной командой (`scripts/deploy.sh`) без автоматического деплоя по CI (раздел 6).

ЗАКЛЮЧЕНИЕ

В ходе разработки создана и доведена до готовности к развёртыванию CMS «Сайтама» – веб-приложение для автоматизации онлайн-записи и операционной деятельности сети барбершопов.

Соответствие целям технического задания. Ключевые измеримые цели (раздел 2.2 технического задания) обеспечены технически: двойное бронирование исключено ограничением целостности СУБД, а не организационно (раздел 8); индивидуальный прайс по мастеру, единое хранилище данных и отчёт по выручке/загруженности без участия разработчика – реализованы и доступны в интерфейсе администратора (раздел 9). Часть целей (снижение доли неявок за счёт напоминаний, доля клиентов, выбирающих вход через VK ID) сформулирована как показатель, измеримый только на реальной эксплуатации.

Не входило в объём технического задания. Второй OAuth-провайдер и другие каналы уведомлений (Telegram, SMS) техническим заданием не предусматривались – вход реализован только по email/паролю и через VK ID (раздел 3), рассылка – только по email (раздел 2). Отдельная таблица журнала аудита критичных операций также не реализована – частично её роль выполняет таблица попыток входа `login_attempts` (раздел 7).

Направления дальнейшего развития. Второй OAuth-провайдер и дополнительные каналы уведомлений можно добавить без изменения архитектуры (раздел 3 уже отделяет бизнес-логику сервисов от конкретного канала доставки/провайдера); горизонтальное масштабирование backend потребует пересмотра внутривещных механизмов защиты от перебора паролей и ограничения частоты запросов (раздел 5) в пользу общего хранилища состояния; полноценный журнал аудита – отдельная таблица с записью каждой критичной операции, а не только попыток входа.

Итоговое состояние проекта, полный перечень функциональных требований и инструкции по эксплуатации – см. соответственно техническое задание, руководство пользователя и руководство администратора.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

- 1) ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – М.: Стандартинформ, 2020.
- 2) ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – М.: Изд-во стандартов, 1978.
- 3) ГОСТ 19.105-78. Единая система программной документации. Общие требования к программным документам. – М.: Изд-во стандартов, 1978.
- 4) ГОСТ Р ИСО/МЭК 15408-3-2008. Информационная технология. Методы и средства обеспечения безопасности. Критерии оценки безопасности информационных технологий. Часть 3. – М.: Стандартинформ, 2008.
- 5) Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных».
- 6) Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации».
- 7) FastAPI – официальная документация [Электронный ресурс]. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 29.08.2026).
- 8) SQLAlchemy 2.0 – официальная документация [Электронный ресурс]. – URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 29.08.2026).
- 9) Alembic – официальная документация [Электронный ресурс]. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 29.08.2026).
- 10) Pydantic – официальная документация [Электронный ресурс]. – URL: <https://docs.pydantic.dev/> (дата обращения: 29.08.2026).
- 11) Vue.js – официальная документация [Электронный ресурс]. – URL: <https://vuejs.org/> (дата обращения: 29.08.2026).
- 12) PostgreSQL 16 – официальная документация [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 29.08.2026).
- 13) Caddy – официальная документация [Электронный ресурс]. – URL: <https://caddyserver.com/docs/> (дата обращения: 29.08.2026).
- 14) Docker Compose – официальная документация [Электронный ре-

сурс]. – URL: <https://docs.docker.com/compose/> (дата обращения: 29.08.2026).

ПРИЛОЖЕНИЕ А. Диаграммы последовательностей

Диаграммы А.1–А.5 иллюстрируют порядок взаимодействия участников для ключевых бизнес-процессов системы, детализированных в карточках функциональных требований технического задания (раздел 4.2).

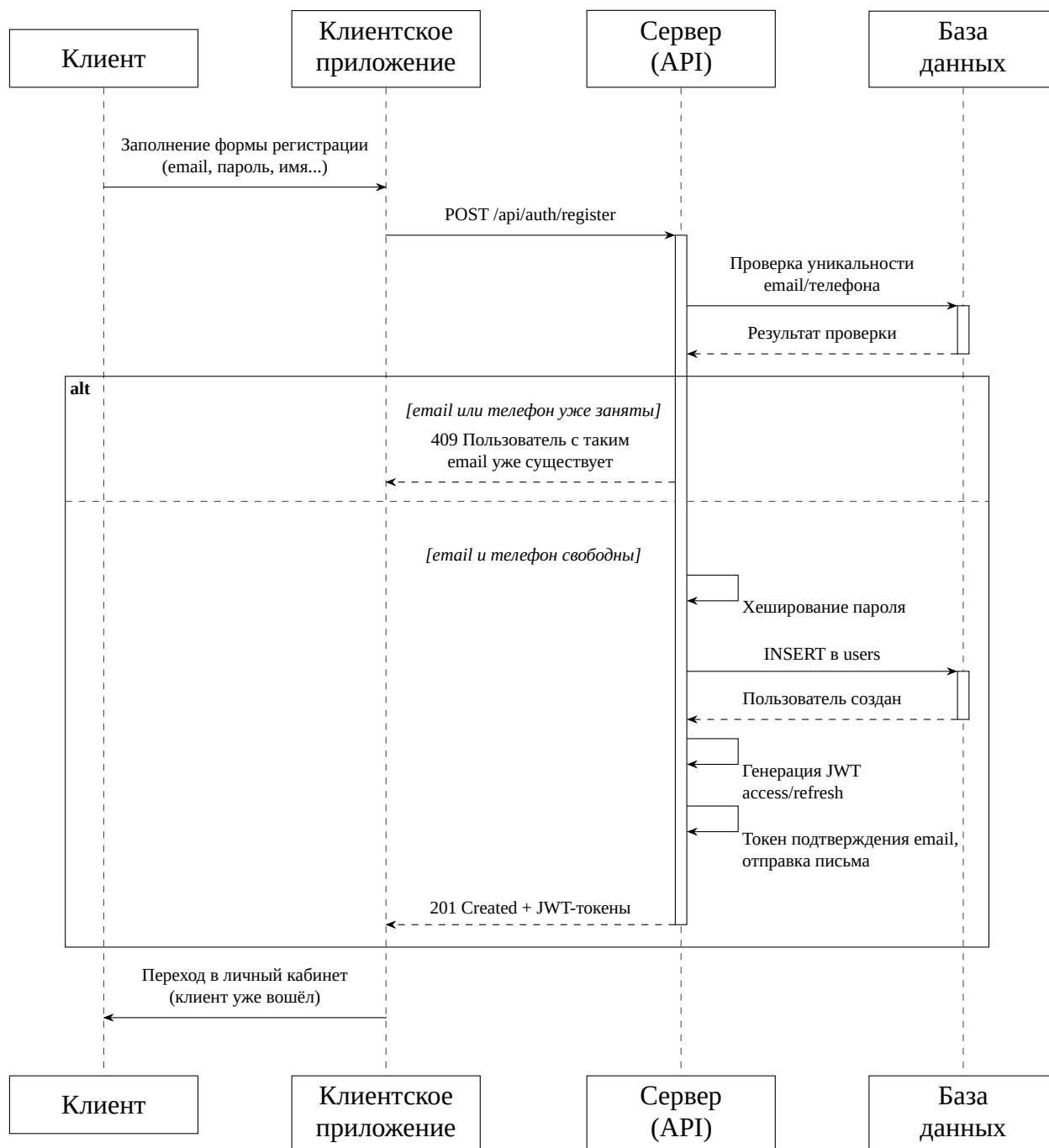


Рисунок А.1 – Диаграмма последовательностей бизнес-процесса «Регистрация» (FR-001)

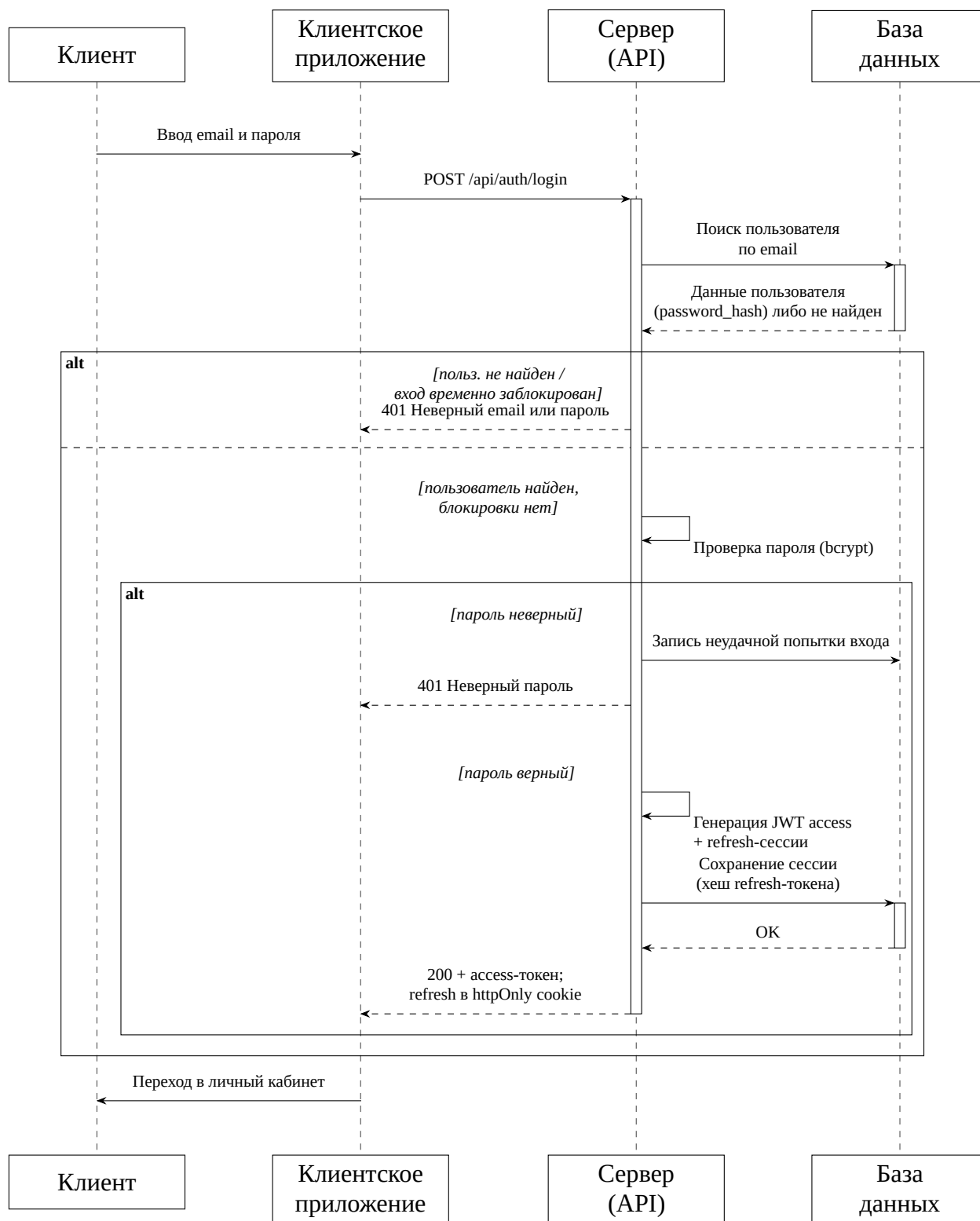


Рисунок А.2 – Диаграмма последовательностей бизнес-процесса «Вход по email и паролю»

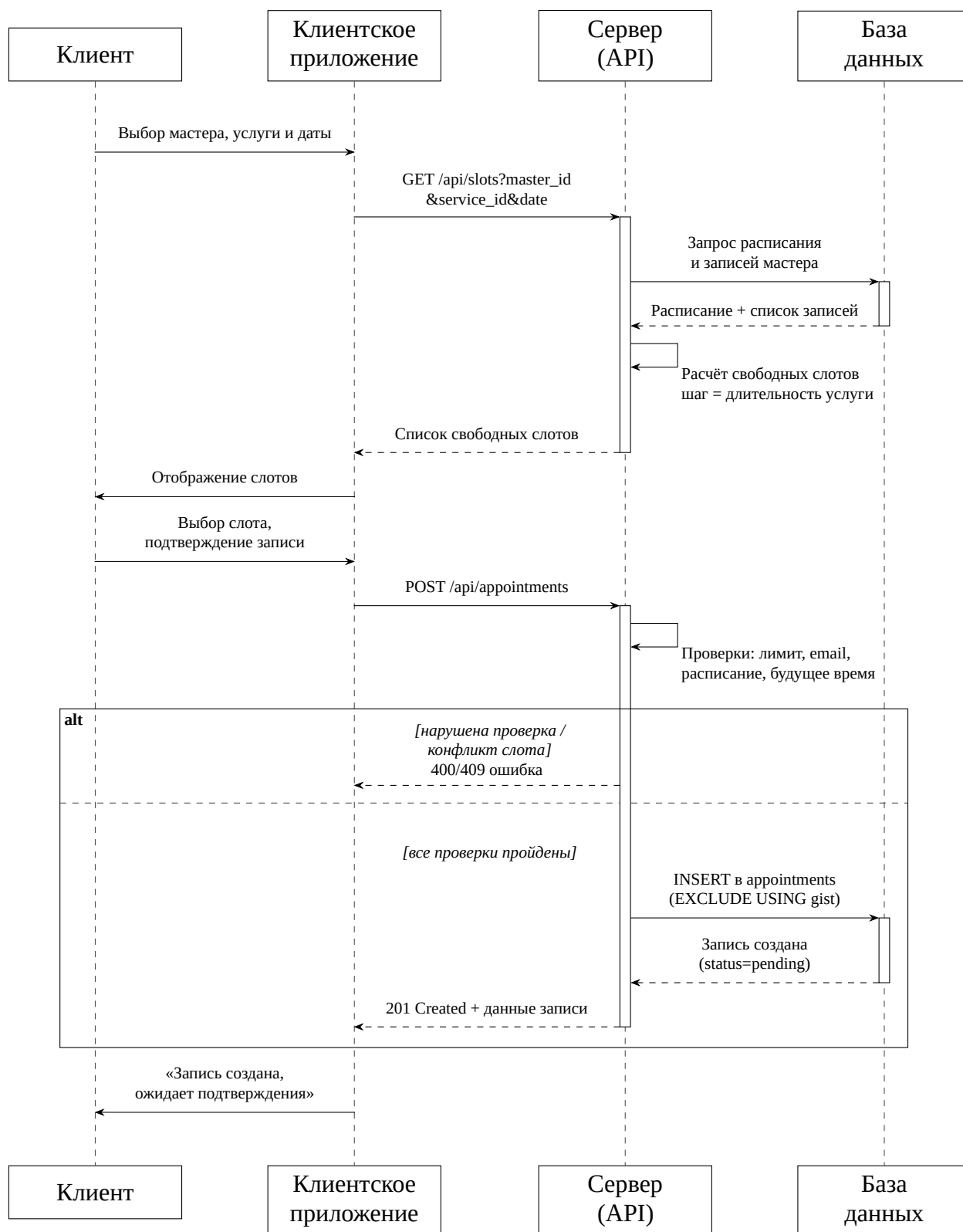


Рисунок А.3 – Диаграмма последовательностей бизнес-процесса «Онлайн-запись на приём» (FR-004, FR-005)

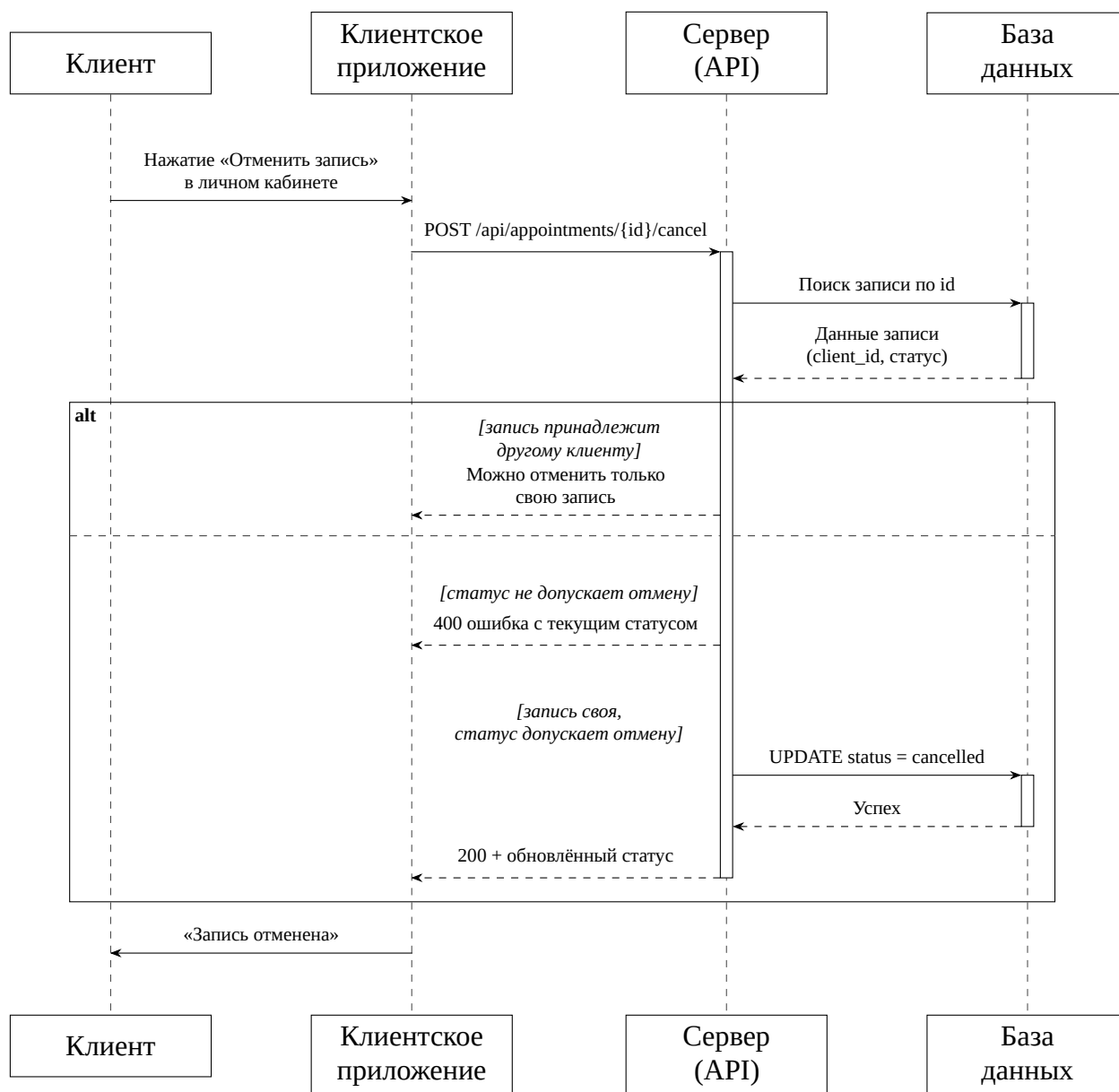


Рисунок А.4 – Диаграмма последовательностей бизнес-процесса «Отмена записи клиентом» (FR-006)

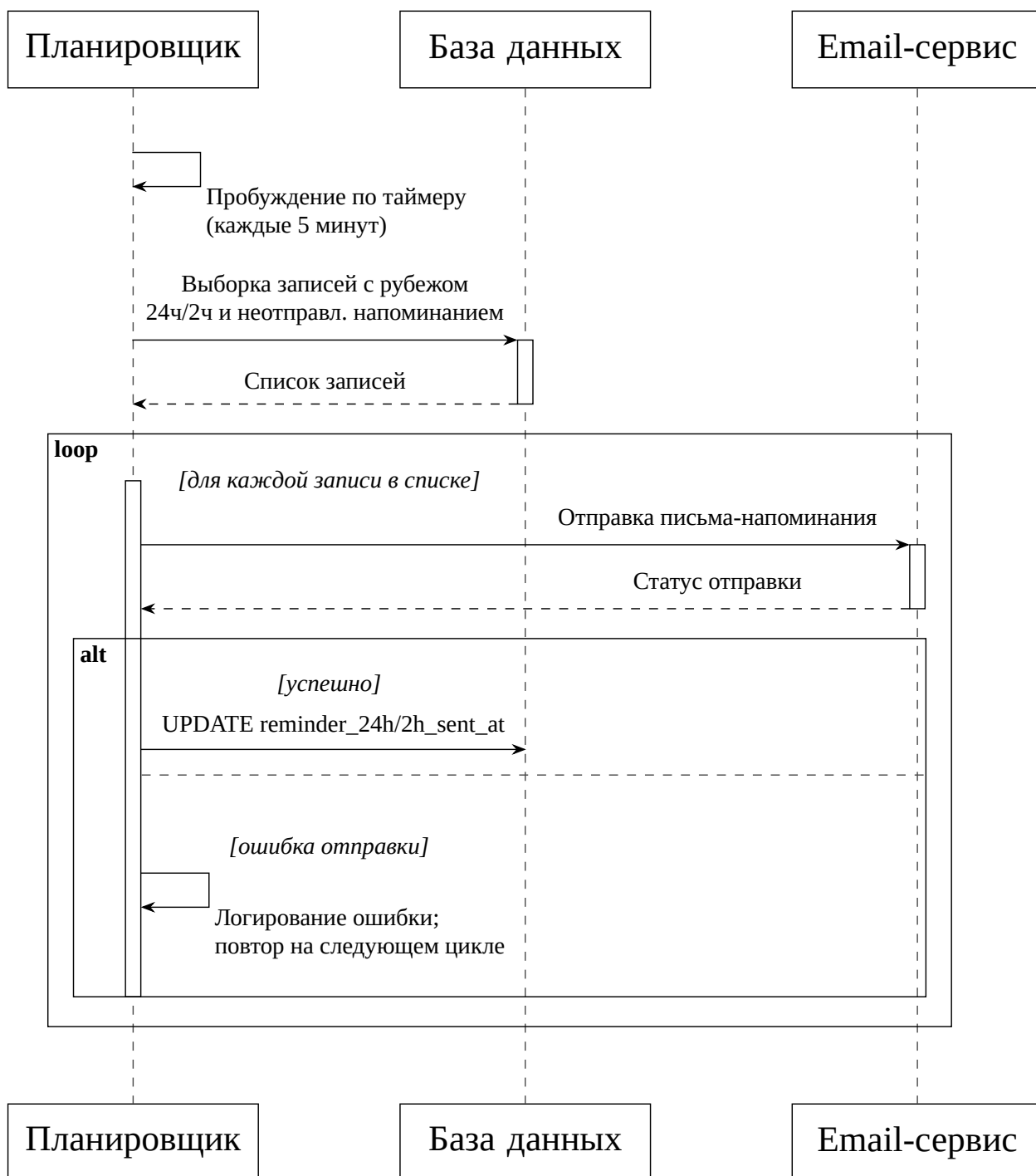


Рисунок А.5 – Диаграмма последовательностей бизнес-процесса
«Автоматическая отправка напоминаний» (FR-008)